SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**Department of Computer Science and Engineering**
**DESIGN AND ANALYSIS OF ALGORITHMS**

## ASSIGNMENT 2 – CO2

**CO2 Coverage:** Brute-Force and Exhaustive-Search techniques – Selection Sort, Bubble Sort, Insertion Sort, Sequential Search, Brute-Force String Matching, Closest Pair, Convex Hull and Exhaustive Search.

**Nature of Assignment:** Individual
**Duration:** 1 Week
**Total Marks:** 50

**NAME:**DEVI SHREE.S

**REGISTER NO:**192511149

**Answer map**

| Q | Algorithm used | Key result |
|---|---|---|
| Q1 | Selection, Bubble, Insertion sort | 95%-ordered data: insertion sort 15,703 comps vs 499,500 (selection) |
| Q2 | Sequential search | Cost = position (hit) or n (miss); boundary n x lambda ~ 2x10^6 |
| Q3 | Brute-force string matching | Same n = 10,000: 9,998 vs 990,100 comparisons |
| Q4 | Brute-force closest pair | n(n-1)/2 pairs; 1 s limit reached near n ~ 3,900 (Python) |
| Q5 | Brute-force convex hull | Student-derived cross-product rule; O(n^3) worst, ~3n^2 typical |
| Q6 | Exhaustive search (airlift knapsack) | 2^n candidates; deploy up to n = 22 (5 s limit) |
| Q7 | Diagnose, redesign (Q6), validate | n = 22: 102 nodes vs 4,194,304 candidates; honest hard case shown |

# Q1 - Sorting Under Different Data Conditions

### 1.1 Prediction before implementation, with algorithmic reason

| Algorithm | Random | Already sorted | Reversed | Reason |
|---|---|---|---|---|
| Selection | n(n-1)/2 comps; <= n-1 swaps | n(n-1)/2 comps; 0 swaps | n(n-1)/2 comps; n/2 swaps | Inner loop always scans the whole unsorted part to find the minimum: comparisons are data-independent. Swaps are at most one per pass. |
| Bubble (with early-exit flag) | ~n(n-1)/2 comps; ~n(n-1)/4 swaps | n-1 comps; 0 swaps | n(n-1)/2 comps and swaps | Each pass compares adjacent pairs; swaps = inversions. The flag stops after a pass with no swap, so sorted input needs one pass. |
| Insertion | ~n^2/4 comps and shifts | n-1 comps; 0 shifts | n(n-1)/2 comps and shifts | Each key moves left past larger keys: shifts = number of inversions I, comparisons = I + (n-1). |

Hence the choice depends on the input: insertion sort (and flagged bubble sort) are adaptive - their work depends on the amount of disorder - whereas selection sort always costs Theta(n^2) comparisons but minimises writes.

### 1.2 Datasets

| Dataset | Role | Construction |
|---|---|---|
| A - Random | Contrasting dataset 1 | Random permutation of 1..1000 (mean of 5 runs) |
| B - Already sorted | Contrasting dataset 2 | 1, 2, ..., 1000 |
| C - Reversed | Edge case of my own (worst case for all three) | 1000, 999, ..., 1 (maximum number of inversions = n(n-1)/2) |
| D - Nearly sorted (95% in place) | Supports the design decision | Sorted list with 25 random disjoint swaps (50 elements = 5% displaced); mean of 5 runs |
| E - 950 sorted + 50 new orders | Supports the design decision | 950 ordered orders followed by a batch of 50 new unsorted orders (the 'small batches added periodically' scenario); mean of 5 runs |
| F - Many duplicates | Extra edge case | 1000 values drawn from only 10 distinct priorities (mean of 5 runs) |

## 1.3 Implementation

```
def selection_sort(a):
    a = a[:]; n = len(a); comps = swaps = 0
    for i in range(n - 1):
        m = i
        for j in range(i + 1, n):
            comps += 1
            if a[j] < a[m]: m = j
```

```
        if m != i: a[i], a[m] = a[m], a[i]; swaps += 1
    return a, comps, swaps

def bubble_sort(a):                    # with early-exit flag
    a = a[:]; n = len(a); comps = swaps = 0
    for p in range(n - 1):
        swapped = False
        for j in range(n - 1 - p):
            comps += 1
            if a[j] > a[j + 1]: a[j], a[j + 1] = a[j + 1], a[j]; swaps += 1; swapped = True
        if not swapped: break
    return a, comps, swaps

def insertion_sort(a):
    a = a[:]; comps = shifts = 0
    for i in range(1, len(a)):
        key = a[i]; j = i - 1
        while j >= 0:
            comps += 1
            if a[j] > key: a[j + 1] = a[j]; shifts += 1; j -= 1
            else: break
        a[j + 1] = key
    return a, comps, shifts
```

### 1.4 Observed results (n = 1000)

| Dataset | Selection comps / swaps | Bubble comps / swaps | Insertion comps / shifts | ms Sel | ms Bub | ms Ins |
|---|---|---|---|---|---|---|
| A Random | 499,500 / 991 | 498,153 / 253,002 | 253,995 / 253,002 | 55.49 | 104.79 | 35.54 |
| B Sorted | 499,500 / 0 | 999 / 0 | 999 / 0 | 33.94 | 0.09 | 0.13 |
| C Reversed | 499,500 / 500 | 499,500 / 499,500 | 499,500 / 499,500 | 35.36 | 96.32 | 117.32 |
| D 95% in place | 499,500 / 25 | 483,895 / 14,704 | 15,703 / 14,704 | 61.11 | 56.62 | 2.32 |
| E 950 sorted + 50 new | 499,500 / 974 | 498,028 / 24,163 | 25,162 / 24,163 | 40.91 | 45.41 | 3.57 |
| F Duplicates | 499,500 / 899 | 493,588 / 227,316 | 228,313 / 227,316 | 37.55 | 68.41 | 35.59 |

Inversion counts (run 0): D = 13,793, E = 23,438. For insertion sort on D: shifts 14,704 + (n - 1) = 15,703 = observed comparisons 15,703: the formula comps = inversions + n - 1 is confirmed exactly.

### 1.5 Prediction vs observation

| Prediction | Observed | Verdict |
|---|---|---|
| Selection: 499,500 comparisons on every input | 499,500 on A, B, C, D, E, F | Confirmed - data-independent |
| Selection swaps <= 999 | A 991, B 0, C 500, D 25 | Confirmed (only one swap per pass) |
| Bubble sorted: 999 comps, 0 swaps | 999 and 0 | Confirmed (early-exit flag) |
| Bubble/Insertion reversed: 499,500 swaps/shifts | 499,500 and 499,500 | Confirmed |
| Insertion random ~ n^2/4 = 250,000 | 253,002 | Confirmed (+1.2%, |

| | | |
|---|---|---|
| shifts | | sampling) |
| Bubble random ~ n(n-1)/4 = 249,750 swaps | 253,002 | Confirmed |
| Insertion on nearly sorted ~ n + inversions | D 15,703, E 25,162 | Confirmed |
| (Not predicted precisely) Bubble on 'nearly sorted' D | 483,895 comps (96.9% of the maximum) | Surprise: see below |

Why bubble sort did NOT benefit on dataset D: the early-exit flag only helps when NO swap occurs in a pass. Bubble sort moves a small element left by one position per pass, so an element displaced by d positions needs d passes. With 25 random swaps, some elements are displaced by hundreds of positions, so ~480 passes are still required. Insertion sort moves each element directly to its place, so its cost is the total displacement (inversions), not the maximum displacement. This is the main algorithmic distinction revealed by the experiment.

### 1.6 Design decision: 95% of orders already in priority order, small batches added periodically Recommendation: INSERTION SORT.

- **Evidence (n = 1,000):** on 95%-ordered data insertion sort needs 15,703 comparisons versus 499,500 (selection, 31.8x more) and 483,895 (bubble, 30.8x more); on the 'sorted + new batch' dataset E it needs 25,162 comparisons (20x fewer than selection/bubble). Time: 2.3 ms vs 61.1 ms (selection).
- **Scaling evidence:** at n = 4,000 on nearly-sorted data insertion needs 61,354 comparisons versus 7,998,000: its cost grows ~ n + I, selection's grows as n^2/2.
- **Why:** cost = n - 1 + I where I is the number of inversions; 95% in place keeps I small. The periodic batch of k new orders adds roughly k*n/2 inversions at most (each new order has to move past some of the existing ones), still far below n^2/4.
- **Where it is not the best:** on random or reversed data insertion sort is as slow as the others (253,995 / 499,500 comparisons) and selection sort writes least (<= n - 1 swaps), which matters only when writes are very expensive. Therefore no algorithm is 'always best'; the input characteristics decide, and this system's characteristic (nearly sorted) favours insertion sort.
- **Properties:** insertion sort is stable (equal priorities keep arrival order - desirable for orders), in place, and online (new orders can be inserted as they arrive).

## Q2 - When Is Sequential Search an Appropriate Engineering Choice?

### 2.1 Experiment design

- Records: (Tool ID, Technician ID, Issue Time, Work Bay) in arrival order; lookup key = Tool ID (algorithm: sequential search, counter on every record comparison).
- List sizes: 10, 50, 100, 500, 1,000, 5,000, 10,000, 50,000 records.
- Successful searches at different positions: first, 25%, 50%, 75%, last; the mean over EVERY present key (exhaustive up to n = 5,000).
- Unsuccessful searches: a Tool ID that is not issued.
- Primary evidence: comparison counts. Supporting evidence: time per search (average of repeated runs) to calibrate cost per comparison.

```
def sequential_search(records, tool_id):
    comps = 0
    for idx, rec in enumerate(records):
        comps += 1                    # one Tool-ID comparison per record
        if rec[0] == tool_id: return idx, comps
    return -1, comps
```

### 2.2 Comparison-count results

| n | First | 25% | 50% | 75% | Last | Absent | Mean (all present keys) | ns / comp. |
|---|---|---|---|---|---|---|---|---|
| 10 | 1 | 3 | 5 | 7 | 10 | 10 | 5.5 | 117 |
| 50 | 1 | 13 | 25 | 37 | 50 | 50 | 25.5 | 96 |
| 100 | 1 | 25 | 50 | 75 | 100 | 100 | 50.5 | 86 |
| 500 | 1 | 125 | 250 | 375 | 500 | 500 | 250.5 | 63 |
| 1,000 | 1 | 250 | 500 | 750 | 1,000 | 1,000 | 500.5 | 69 |
| 5,000 | 1 | 1,250 | 2,500 | 3,750 | 5,000 | 5,000 | 2,500.5 | 78 |
| 10,000 | 1 | 2,500 | 5,000 | 7,500 | 10,000 | 10,000 | 5,000.5 | 138 |
| 50,000 | 1 | 12,500 | 25,000 | 37,500 | 50,000 | 50,000 | 25,000.5 | 101 |

- **Successful search:** the cost equals the 1-indexed position of the record (1, n/4, n/2, 3n/4, n) exactly, and the mean over all keys is (n+1)/2 (e.g. 500.5 for n = 1,000: verified exhaustively).
- **Unsuccessful search:** always exactly n comparisons - the worst case, and twice the average successful cost. It matters operationally: a technician asking about a tool that has just been returned (so it is no longer in the list) pays the full scan.
- **Size:** counts grow linearly (x10 records = x10 comparisons). Measured cost per comparison ~ 96-101 ns (rising slightly with n as the list stops fitting in cache); I use c = 100 ns as a conservative figure.

### 2.3 Operating conditions: when is sequential search reasonable, and when do I stop recommending it?

Acceptability criteria chosen: (i) worst-case latency of one search <= 100 ms (interactive), and (ii) the searches must consume no more than 10% of one CPU core. With c = 100 ns, request rate lambda (searches/second) and n records:

latency (worst) = n * c <= 0.1 s  =>  n <= 10^6 records

load (average) = lambda * (n/2) * c <= 0.10  =>  n * lambda <= 0.2 / c = 2 x 10^6

Sequential search is therefore a reasonable engineering choice when n*lambda stays well below ~2x10^6 comparisons-worth of demand (n up to ~10^4-10^5 records at a modest request rate), when the list is changing continuously (issue = append in O(1); return = delete; no ordering or index to maintain), and when simplicity matters. I stop recommending it when (a) n*lambda exceeds roughly 10^5-10^6, (b) n exceeds ~10^5 (latency becomes noticeable and the unsuccessful-search cost n becomes unpredictable), or (c) each comparison becomes expensive (network/database access, ~100x the 100 ns assumed here), which divides every threshold by the same factor.

| Scenario (assumptions stated) | n | lambda | Worst latency | CPU load | n x lambda | % of 2x10^6 limit |
|---|---|---|---|---|---|---|
| Original: one hangar, ~300 issued tools, 30 look-ups/min | 300 | 0.5/s | 30 us | 0.00075% | 150 | 0.0075% |
| 20 facilities, one central list: 20x records and 20x requests | 6,000 | 10/s | 0.6 ms | 0.30% | 60,000 | 3% |
| 20 facilities growing 10x (60,000 records, 100 requests/s) | 60,000 | 100/s | 6 ms | 30% | 6,000,000 | 300% - beyond the limit |

### 2.4 Changed condition: 20 maintenance facilities - re-evaluation

- **Result of the re-evaluation:** on the evidence, a single sequential scan of a central list is still affordable at the assumed 20-facility load (3% of the limit, 0.6 ms worst latency), so sequential search is NOT immediately unacceptable. The boundary is crossed when the central list grows by another ~30x (n x lambda ~ 2x10^6), which the 20-fold expansion makes plausible within a few years.
- **What changes the engineering judgement:** demand grows as n x lambda, i.e. the product of two quantities that both grew 20x (x400 total, from 150 to 60,000). A federated lookup 'which facility has T-184?' that scans 20 separate lists costs the sum of all list lengths for an absent or last-position tool.
- **Recommendation:** retain sequential search for the per-hangar lists (n ~ 300, cost 30 microseconds - an index would cost more than it saves); for the consolidated 20-facility system replace it with a hash index keyed by Tool ID - the list changes continuously, so a sorted array (O(n) shifting per issue/return) would be a poor choice, whereas a hash table gives O(1) average search, insert (issue) and delete (return). Adopt the index when n x lambda reaches ~10^5 (5% of the limit) or when a response-time guarantee is required, rather than waiting for failure.
- **Evidence-based operating boundary:** n x lambda <= ~10^5-10^6 and n <= ~10^5 with comparison cost ~100 ns: sequential search recommended; beyond: stop recommending.

# Q3 - Engineer the Behaviour of Brute-Force String Matching

## 3.1 Algorithm

```
ALGORITHM BruteForceMatch(T[0..n-1], P[0..m-1])
for i <- 0 to n-m do                    // each alignment (shift)
   j <- 0
   while j < m and T[i+j] = P[j] do     // character comparison (basic operation)
      j <- j + 1
   if j = m then return i               // first match
return -1
```

```
def brute_force_match(text, pattern):
   n, m = len(text), len(pattern); comps = 0
   for i in range(n - m + 1):
      j = 0
      while j < m:
         comps += 1
         if text[i + j] != pattern[j]: break
         j += 1
      if j == m: return i, comps
   return -1, comps
```

## 3.2 Three test conditions, predictions (made before running) and reasons

| Test | Text / pattern | Prediction before execution | Algorithmic reason |
|---|---|---|---|
| 1 - little work | text = 'abcdefghij' x 1000 (n = 10,000); pattern 'XYZ' (m = 3), absent | n - m + 1 = 9,998 comparisons (about 1 per alignment) | The first pattern character never matches any text character, so every alignment is rejected after ONE comparison. |
| 2 - engineered for many comparisons | text = 'A' x 9,999 + 'B' (n = 10,000); pattern 'A' x 99 + 'B' (m = 100) | (n - m + 1) x m = 9,901 x 100 = 990,100 comparisons (almost n x m) | At every alignment the first 99 characters match and only the LAST pattern character fails: 100 comparisons per alignment and no information is reused when the pattern shifts by one. |
| 3 - my own: realistic security log | sshd log of 1,200 lines (n = 113,142 chars, 594 'Failed password for' lines); pattern 'Failed password for invalid user' (m = 32), absent | (n - m + 1) + 20 x 594 = 124,991 | Most alignments fail at the first character (cost 1); only alignments starting at a 'Failed password for' line match the 20-character prefix and fail on the 21st character (cost 21 = 1 + 20 extra). |

## 3.3 Observed character-comparison evidence

| Test | n | m | Match index | Comparisons (observed) | Predicted | comps / n | comps / (n x m) | Time (ms) |
|---|---|---|---|---|---|---|---|---|
| 1 Easy | 10,000 | 3 | none | 9,998 | 9,998 | 1.000 | 0.333 | 1.22 |
| 2 Engineered | 10,000 | 100 | 9900 | 990,100 | 990,100 | 99.01 | 0.9901 | 89.28 |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 3 Security log | 113,142 | 32 | none | 124,991 | 124,991 | 1.105 | 0.0345 | 14.54 |

- Predictions were exact: Test 1 and Test 2 match to the last digit; Test 3 matches exactly (124,991) once the comparison at which the partial match fails is counted (my first estimate of +19 per line omitted that failing 21st comparison; the corrected reasoning above predicts the observed value exactly).
- Extra evidence on match position: a pattern equal to the first 25 characters of the log is found after only 25 comparisons (index 0), while the last 25 characters need 115,742 (the whole text must be scanned).

| m | Worst-case comps (n = 10,000) | Formula (n-m+1) x m |
|---|---|---|
| 5 | 49,980 | 49,980 |
| 10 | 99,910 | 99,910 |
| 20 | 199,620 | 199,620 |
| 50 | 497,550 | 497,550 |
| 100 | 990,100 | 990,100 |
| 200 | 1,960,200 | 1,960,200 |
| 400 | 3,840,400 | 3,840,400 |

### 3.4 Inquiry

- **Why one input requires substantially more work than another:** identical n = 10,000, but 9,998 vs 990,100 comparisons (99x). In Test 1 each alignment is rejected immediately (mismatch on the first character); in Test 2 each alignment is rejected only at the last character, so all m characters are compared and the work is thrown away when the pattern shifts by one.
- **Which characteristics create repeated comparisons:** (i) a pattern with a long prefix that occurs repeatedly in the text (partial matches), (ii) a text with long runs of the same character(s) as the pattern, (iii) mismatch at the END of the pattern rather than the start, (iv) long patterns (the extra work per bad alignment is up to m-1). Real logs create a milder form of this: lines sharing the prefix 'Failed password for ' cost 21 instead of 1 comparisons.
- **What this reveals about evaluating an algorithm by input size alone:** n alone cannot predict the cost - the running time ranges from ~n to ~n x m for the same n, i.e. Theta(n) best vs Theta(nm) worst. A meaningful evaluation needs the sizes n AND m and the structure/content of the data, hence best/average/worst-case analysis and tests designed around structure rather than only larger inputs.

## Q4 - Closest Pair and the Cost of Examining Every Possibility

### 4.1 Design before implementation

- **Which candidate pairs must be examined:** every unordered pair of distinct drones {D_i, D_j}, because brute force has no information excluding any pair; the closest pair could be any of them.
- **How duplicate comparisons are avoided:** distance is symmetric (d(i,j) = d(j,i)), so examine pair (i, j) only for j > i; self-pairs (i = i) are excluded. We also compare squared distances dx^2 + dy^2 (no square root needed to find the minimum; the root is taken once at the end).
- **How many unique pairs exist:** derived below.

  Pairs = sum_{i=1}^{n-1} (n - i) = (n-1) + (n-2) + ... + 1 = n(n-1)/2 = C(n,2)

  Check: choose 2 of n = n(n-1)/2 ordered selections / 2! (order irrelevant) -> n=100: 4,950; n=1,000: 499,500; n=10^6: 4.99999 x 10^11

```
ALGORITHM BruteForceClosestPair(D[0..n-1])
best <- infinity
for i <- 0 to n-2 do
   for j <- i+1 to n-1 do                    // each unordered pair once
      d <- (x_i - x_j)^2 + (y_i - y_j)^2      // distance calculation (basic operation)
      if d < best then best <- d ; (a,b) <- (i,j)
return (a, b, sqrt(best))
```

```
def closest_pair_counted(pts):
   n = len(pts); best = float("inf"); pairs = calcs = 0
   for i in range(n - 1):
      xi, yi = pts[i]
      for j in range(i + 1, n):
         pairs += 1
         dx = xi - pts[j][0]; dy = yi - pts[j][1]
         d = dx*dx + dy*dy; calcs += 1
         if d < best: best, bi, bj = d, i, j
   return (bi, bj, math.sqrt(best)), pairs, calcs
```

### 4.2 Experimental results (random drone positions in a 1000 x 1000 area)

| n | Pairs examined | Theory n(n-1)/2 | Distance calcs | Time (s) | ns per n^2 | Closest distance |
|---|---|---|---|---|---|---|
| 100 | 4,950 | 4,950 | 4,950 | 0.0006 | 55.7 | 7.519 |
| 200 | 19,900 | 19,900 | 19,900 | 0.0022 | 53.8 | 3.338 |
| 400 | 79,800 | 79,800 | 79,800 | 0.0092 | 57.3 | 0.576 |
| 800 | 319,600 | 319,600 | 319,600 | 0.0392 | 61.2 | 0.856 |
| 1,600 | 1,279,200 | 1,279,200 | 1,279,200 | 0.1487 | 58.1 | 0.674 |
| 3,200 | 5,118,400 | 5,118,400 | 5,118,400 | 0.6745 | 65.9 | 0.195 |
| 6,400 | 20,476,800 | 20,476,800 | 20,476,800 | 2.7419 | 66.9 | 0.029 |

The count of pairs examined and the number of distance calculations equal n(n-1)/2 exactly for all seven sizes (n = 100 ... 6,400), confirming the derivation. The results were also checked against an independent min() over itertools.combinations for n <= 800. Timing is supporting evidence (separate run without counters).

### 4.3 Theoretical growth vs experiment

Theory: pairs = Theta(n^2) -> doubling n multiplies the work by 4 (n^2 coefficient 1/2). Observed: pairs ratio per doubling = 4.001 (3200 -> 6400); time ratio = 4.07 (4.3 here; slightly above 4 because

larger point arrays no longer fit in the CPU cache). The fitted constant is t = 6.30e-08 n^2 seconds (about 126 ns per pair).

### 4.4 Scalability inquiry

- **Acceptable response time (my definition):** positions are reported about once per second, so the collision check must finish within 1 second (before the next snapshot); 10 seconds is the absolute outer limit for a safety alert.
- **When brute force becomes impractical:** from t = 6.30e-08 n^2: n = sqrt(1 / k) = 3,983 drones for 1 s, n = 12,596 for 10 s in this Python environment. Measured: n = 3,200 takes 0.67 s (acceptable), n = 6,400 takes 2.74 s (> 1 s, impractical for per-second monitoring). A compiled implementation would be roughly 50-100x faster per pair, moving the boundary to ~30,000-40,000 drones (an estimate, not measured here) - but the n^2 wall remains: at 10^6 drones there are 5 x 10^11 pairs.
- **Where the inefficiency originates:** every one of the n(n-1)/2 pairs is evaluated independently, although the vast majority are far apart and cannot be the closest pair; the algorithm uses neither the spatial layout of the drones nor the best distance found so far, and it repeats the whole computation at every snapshot even though most drones have barely moved. The cost is quadratic in the NUMBER OF CANDIDATE PAIRS, not in the amount of information in the problem.
- **What a redesign would need to avoid or reduce:** (i) examining pairs that are provably farther apart than the current best distance (use spatial ordering/partitioning so that only nearby drones are compared), (ii) examining all n(n-1)/2 pairs rather than a number proportional to n or n log n, and (iii) recomputing from scratch every second (reuse the previous result and re-check only drones that moved). Not implemented, as the question permits.

## Q5 - Derive a Convex Hull Algorithm from a Geometric Principle

### 5.1 Converting the geometric principle into a computational procedure

Principle given: a line joining two candidate points may form part of the outer boundary if all remaining points lie on one appropriate single side of that line. My derivation:

- **How two candidate points define a line:** points P1 = (x1, y1) and P2 = (x2, y2) define the line a x + b y = c with a = y2 - y1, b = x1 - x2, c = x1 y2 - y1 x2. Direct it from P1 to P2 so that 'side' can be told apart.
- **How the position of another point relative to the line is determined:** evaluate the sign of s(P) = a x + b y - c for P = (x, y): s > 0 on one side, s < 0 on the other, s = 0 exactly on the line. Equivalent cross-product form (used in the code, with the sign flipped): cross(P1, P2, P) = (x2 - x1)(y - y1) - (y2 - y1)(x - x1) = -(a x + b y - c); cross > 0 means P is to the LEFT of the directed line P1->P2, cross < 0 means right, 0 collinear.
- **How the remaining points are examined:** test all other n - 2 points; reject the candidate pair as soon as one point lies strictly on the wrong (right) side (early exit); a point on the line is acceptable only if it lies between P1 and P2 (otherwise P1P2 is not an extreme segment).
- **Condition identifying a potential hull edge:** directed segment P_i -> P_j is a hull edge iff cross(P_i, P_j, P_k) >= 0 for every other point P_k (and every collinear P_k lies between P_i and P_j). Choosing 'left side' fixes counter-clockwise orientation, so each edge is accepted in exactly one direction.
- **How the final boundary is constructed:** store each accepted edge as next[i] = j; start at the lowest-leftmost hull vertex and follow next[] until the start is reached - this gives the vertices in counter-clockwise order. Collinear points inside an edge are not vertices.

### 5.2 Pseudocode

```
ALGORITHM BruteForceHull(P[0..n-1])            // distinct points
edges <- empty ; tests <- 0
for i <- 0 to n-1 do
  for j <- 0 to n-1, j != i do                 // ORDERED pair = directed line P_i -> P_j
     ok <- true
     for k <- 0 to n-1, k != i, k != j do
        c <- (xj - xi)*(yk - yi) - (yj - yi)*(xk - xi)     // cross product, 1 test
        if c < 0 then ok <- false ; break                  // point on the wrong (right) side
        if c = 0 and P_k not between P_i and P_j then ok <- false ; break
     if ok then add edge (i -> j) to edges ; next[i] <- j
start <- vertex with smallest (x, y) among edges ; hull <- [start]
v <- next[start]
while v != start do append v to hull ; v <- next[v]
return hull                                    // counter-clockwise
```

```
def cross(p, q, r): return (q[0]-p[0])*(r[1]-p[1]) - (q[1]-p[1])*(r[0]-p[0])
def on_segment(p, q, r): return min(p[0],q[0]) <= r[0] <= max(p[0],q[0]) and min(p[1],q[1]) <= r[1] <= max(p[1],q[1])
def hull_bruteforce(P, early_exit=True):
    n = len(P); edges = []; tests = 0
    for i in range(n):
        for j in range(n):
            if i == j: continue
            ok = True
            for k in range(n):
                if k == i or k == j: continue
                tests += 1
                c = cross(P[i], P[j], P[k])
```

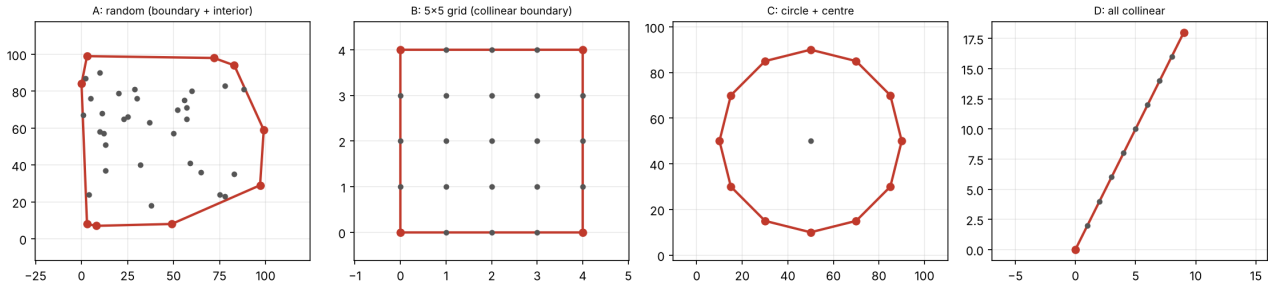
```
            if c < 0 or (c == 0 and not on_segment(P[i], P[j], P[k])):
                ok = False
                if early_exit: break
        if ok: edges.append((i, j))
    ... # chain edges via next[] (see Appendix A)
```

### 5.3 Tests: boundary + interior points, and configurations designed to challenge the algorithm

| Point set | n | Hull vertices | Accepted edges | Orientation tests (early exit) | n(n-1)(n-2) |
|---|---|---|---|---|---|
| A_random_boundary_and_interior | 40 | 9 | 9 | 5,036 | 59,280 |
| B_5x5_grid_collinear | 25 | 4 | 4 | 2,674 | 13,800 |
| C_circle_plus_centre | 13 | 12 | 12 | 521 | 1,716 |
| D_all_collinear | 10 | 2 | 2 | 160 | 720 |

- Set A (40 random stations): both boundary and interior stations; 9 hull vertices found.
- Set B (5 x 5 grid): CHALLENGE 1 - collinear stations lie on every boundary side and many points are collinear with candidate lines; the algorithm correctly returns only the 4 corners (collinear points inside an edge are accepted as lying 'between' and are not vertices).
- Set C (12 points on a circle + centre): CHALLENGE 2 - almost every point is a hull vertex, the centre must be rejected; 12 vertices found.
- Set D (10 collinear points): degenerate - hull is the two extreme points.
- Every hull was compared with an independent reference (monotone-chain algorithm): identical vertex sets in all 4 sets and in all 6 random growth sets (assertions in the code).

Graph Q5-1: hull boundary found by the brute-force test (red = hull vertices/edges, grey = other stations)




Hull of set A (CCW): [[0, 84], [3, 8], [8, 7], [49, 8], [97, 29], [99, 59], [83, 94], [72, 98], [3, 99]]
Hull of set B (CCW): [[0, 0], [4, 0], [4, 4], [0, 4]]

### 5.4 Analysis of computational work as n grows

| n | Hull size h | Tests, early exit | Tests / n^2 | Tests, no early exit | = n(n-1)(n-2) | Time early (s) | Time full (s) |
|---|---|---|---|---|---|---|---|
| 10 | 5 | 222 | 2.22 | 720 | 720 | 0.0001 | 0.000 |
| 20 | 8 | 1,049 | 2.62 | 6,840 | 6,840 | 0.0004 | 0.003 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 40 | 10 | 5,438 | 3.40 | 59,280 | 59,280 | 0.0020 | 0.022 |
| 80 | 9 | 20,210 | 3.16 | 492,960 | 492,960 | 0.0079 | 0.183 |
| 160 | 12 | 80,837 | 3.16 | 4,019,520 | 4,019,520 | 0.0317 | 1.417 |
| 320 | 13 | 305,630 | 2.98 | 32,461,440 | 32,461,440 | 0.1488 | 13.974 |

| n (all points on hull) | Hull size | Tests (early exit) | n(n-1)(n-2) | Tests / n^3 |
|---|---|---|---|---|
| 10 | 10 | 272 | 720 | 0.272 |
| 20 | 20 | 1,842 | 6,840 | 0.230 |
| 40 | 40 | 12,882 | 59,280 | 0.201 |
| 80 | 80 | 94,562 | 492,960 | 0.185 |
| 160 | 160 | 720,322 | 4,019,520 | 0.176 |

- **Theory:** n(n-1) ordered pairs x (n-2) tests = n(n-1)(n-2) = Theta(n^3) tests in the worst case. Without early exit this is exactly what is observed (e.g. 32,461,440 tests for n = 320, 13.1 s).
- **With early exit on random points:** almost every non-hull pair is rejected after ~3 tests (about half of the points lie on the wrong side), so the work is ~3 n^2 (tests / n^2 = 2.98 at n = 320; n x 2 -> work x 4). Only the h x (n - 2) tests for true hull edges are unavoidable.
- **Worst case persists:** when all points are hull vertices the early exit rarely helps: tests / n^3 falls from 0.27 (n = 10) to 0.185 (n = 80) and 0.176 (n = 160) and is flattening, consistent with Theta(n^3) (n = 160: 720,322 tests). So the correct statement is O(n^3) worst case, ~n^2 typical for random points.

### 5.5 Correctness inquiry: why does the test show an accepted edge belongs to the outer boundary?

- **Sufficiency (accepted => boundary):** if all points lie in the closed half-plane H on the left of line L through P_i and P_j, then, because a half-plane is convex, the convex hull of the points (the smallest convex set containing them) is also inside H. Hence no point or hull point lies beyond L, and L passes through two hull points P_i, P_j (they belong to the set). A line that touches the set and has all of it on one side is a supporting line; the part of the hull on a supporting line is a vertex or an edge. Since L contains the two distinct points P_i, P_j and every other point on L lies between them, the segment P_iP_j lies on the hull boundary - it is exactly a hull edge (its endpoints are extreme points).
- **Why a point on the wrong side rejects the edge:** if some P_k is strictly on the right of P_i->P_j, then the hull contains points on both sides of L, so the interior of the hull crosses L: L cuts through the interior and the segment lies partly inside the polygon - it is a chord (diagonal), not boundary.
- **Why collinear points are handled this way:** a point on L outside segment P_iP_j means P_i or P_j is not an extreme point (the true extreme is further along L); rejecting such pairs ensures only extreme endpoints are accepted.
- **Completeness (boundary => accepted):** every hull edge has a supporting line with all points on one side, so it is accepted in the CCW direction by the same test; reversed direction fails unless all points are collinear. Hence the accepted edges are exactly the hull edges and chaining them yields the boundary.

- **Evidence beyond the argument:** sets A-D and 6 random sets matched an independent algorithm, including the collinear and all-on-hull challenges.

# Q6 - Exhaustive Search and Combinatorial Growth

## 6.1 Exhaustive-search strategy (designed from scratch)

| Question | Design decision |
|---|---|
| How candidate combinations are represented | A bit vector x = (x_1, ..., x_n), x_i in {0, 1}: x_i = 1 means package i is loaded. Every subset of the n packages corresponds to exactly one vector, so the solution space has 2^n elements. |
| How candidate solutions are generated | Recursive include/exclude enumeration (a binary decision tree of depth n): at package i first include it, then exclude it; every leaf is one complete candidate. (Equivalent: count masks 0 ... 2^n - 1.) |
| How feasibility is checked | At each leaf compute total weight = sum w_i x_i and accept the candidate only if total weight <= payload limit C. |
| How feasible solutions are evaluated | total value = sum v_i x_i for feasible candidates only. |
| How the best solution is retained | Keep (best value, best vector); replace it when a feasible candidate has strictly higher value (ties keep the first found). |

```
ALGORITHM ExhaustiveAirlift(w[1..n], v[1..n], C)
best <- -1 ; bestVec <- none
procedure Enumerate(i, weight, value, x)
   if i = n+1 then                          // complete candidate
      examined <- examined + 1
      if weight <= C then                   // feasibility check
         feasible <- feasible + 1
         if value > best then best <- value ; bestVec <- copy of x   // retain best
      return
   x[i] <- 1 ; Enumerate(i+1, weight + w[i], value + v[i], x)   // include package i
   x[i] <- 0 ; Enumerate(i+1, weight,        value,        x)   // exclude package i
Enumerate(1, 0, 0, x)
return best, bestVec
```

```
def exhaustive_knapsack(W, V, cap):
   n = len(W); stats = {"examined": 0, "feasible": 0}; best = [-1, None]; chosen = [0]*n
   def rec(i, w, v):
      if i == n:
         stats["examined"] += 1
         if w <= cap:
            stats["feasible"] += 1
            if v > best[0]: best[0] = v; best[1] = chosen[:]
         return
      chosen[i] = 1; rec(i+1, w + W[i], v + V[i])
      chosen[i] = 0; rec(i+1, w, v)
   rec(0, 0, 0)
   return best[0], best[1], stats
```

## 6.2 Correctness check on a small instance verified independently

Instance: 4 packages P1..P4 with weights (4, 3, 2, 5), values (40, 30, 25, 60), payload limit 9. All 2^4 = 16 combinations, listed by hand-checkable arithmetic:

| x1x2x3x4 | Packages | Weight | Value | Feasible? |
|---|---|---|---|---|
| 0000 | {} | 0 | 0 | feasible |

| | | | | |
|---|---|---|---|---|
| 1000 | {P1} | 4 | 40 | feasible |
| 0100 | {P2} | 3 | 30 | feasible |
| 1100 | {P1,P2} | 7 | 70 | feasible |
| 0010 | {P3} | 2 | 25 | feasible |
| 1010 | {P1,P3} | 6 | 65 | feasible |
| 0110 | {P2,P3} | 5 | 55 | feasible |
| 1110 | {P1,P2,P3} | 9 | 95 | feasible |
| 0001 | {P4} | 5 | 60 | feasible |
| 1001 | {P1,P4} | 9 | 100 | feasible |
| 0101 | {P2,P4} | 8 | 90 | feasible |
| 1101 | {P1,P2,P4} | 12 | 130 | too heavy |
| 0011 | {P3,P4} | 7 | 85 | feasible |
| 1011 | {P1,P3,P4} | 11 | 125 | too heavy |
| 0111 | {P2,P3,P4} | 10 | 115 | too heavy |
| 1111 | {P1,P2,P3,P4} | 14 | 155 | too heavy |

The table shows 12 feasible and 4 overweight combinations; the best feasible value is 100 with {P1, P4} (weight 9 = limit exactly). The exhaustive algorithm returned value 100, vector [1, 0, 0, 1], examined = 16, feasible = 12; an independent solver based on itertools.combinations gave the same optimum. Further checks: for random instances with n = 6, 8, 10, 12, 14 the optimum matched the independent solver in every case (assertions in the code). Verdict: the algorithm identifies the correct optimum.

### 6.3 Growth investigation - prediction before experimentation

- **Prediction of the number of candidate combinations:** each package has two states, so the count is the product 2 x 2 x ... x 2 = 2^n: n = 10 -> 1,024; n = 20 -> 1,048,576; n = 30 -> 1,073,741,824.
- **Prediction when packages are added:** each additional package DOUBLES the candidates (and approximately the time); adding 10 packages multiplies the work by 2^10 = 1,024; adding 20 by about a million. Time per candidate is constant, so time ~ 0.38 us x 2^n.

### 6.4 Experimental validation (random weights 5-50, values 10-100, payload limit = 40% of total weight)

| n packages | Predicted 2^n | Candidates examined | Feasible | Feasible % | Best value | Time (s) |
|---|---|---|---|---|---|---|
| 8 | 256 | 256 | 86 | 33.6% | 239 | 0.0001 |
| 10 | 1,024 | 1,024 | 297 | 29.0% | 269 | 0.0003 |
| 12 | 4,096 | 4,096 | 1,096 | 26.8% | 442 | 0.0014 |
| 14 | 16,384 | 16,384 | 4,096 | 25.0% | 514 | 0.0058 |
| 16 | 65,536 | 65,536 | 15,521 | 23.7% | 570 | 0.0222 |
| 18 | 262,144 | 262,144 | 57,515 | 21.9% | 566 | 0.0901 |
| 20 | 1,048,576 | 1,048,576 | 228,800 | 21.8% | 654 | 0.5323 |
| 22 | 4,194,304 | 4,194,304 | 876,241 | 20.9% | 917 | 1.9885 |

### 6.5 Inquiry: why can a few additional packages cause a dramatic increase in effort?

The effort is proportional to the size of the solution space, which is the PRODUCT of independent choices, 2^n, not the sum. Adding one package does not add one more candidate; it doubles the whole space, because every existing combination can now appear with and without the new package. Thus the running time grows exponentially: n -> n + 10 multiplies the work by 1,024 (1.6 s at n = 22 becomes about 27 minutes at n = 32). Because exhaustive search must visit every leaf, no amount of faster hardware helps for long: a machine 1,000 times faster buys only 10 more packages.

### 6.6 Deployment decision

- **Response-time limit (selected and justified):** 5 seconds - a loading decision for an emergency airlift is interactive and must be produced while the helicopter is being prepared; a delay longer than a few seconds is operationally unacceptable.
- **Largest problem size:** 2^n x 0.474 us <= 5 s => n <= log2(5 / 4.741e-07) = 23.3 => n = 23 at the limit (predicted 3.2 s at n = 23, 6.3 s at n = 24 - already above the limit). Measured: n = 22 took 1.99 s. With a safety margin for slower hardware I would deploy the exhaustive-search solution for at most n = 22 packages.
- **Practical meaning:** beyond ~22 candidate packages (or on slower devices), the exhaustive solution must not be used; a compiled implementation could gain about 5-6 more packages (x50 faster = log2 50 = 5.6) but not 20.

## Q7 - Design Challenge: Diagnose, Redesign and Validate

Selected solution: the Q6 exhaustive airlift search (clear exponential limitation: 2^n candidates, measured above). The redesign below is developed from the diagnosis; it is not a replacement by a named textbook algorithm.

**A. Diagnose the Computational Bottleneck**

- **Where most computational effort is spent:** in generating and testing complete candidates at the leaves: 2^n leaves = 4,194,304 at n = 22 (1.99 s). Per-candidate work is constant (0.376 us), so effort = number of candidates.
- **Which operation grows significantly with the input:** the number of candidate examinations (feasibility test + value computation), which doubles with every added package (2^n; measured 256 -> 4,194,304 from n = 8 to 22).
- **Whether work is unnecessarily repeated:** YES. Feasibility is checked only at the leaves, so once a partial selection already exceeds the payload limit, all 2^(n-i) completions of that prefix are still generated and then rejected one by one: 3,318,063 of 4,194,304 candidates (79.1%) at n = 22 are infeasible (75.0% at n = 14, 78.1% at n = 18, 79.1% at n = 22 - the wasted share is rising). In addition, candidates that cannot beat the best value found so far are fully evaluated.
- **Which characteristic of the original algorithm causes the limitation:** it is a blind generate-and-test search: constraints and objective are consulted only AFTER a complete candidate exists, so the structure of the problem (weights only increase as items are added; each item has a known value-to-weight ratio) is never used to discard whole groups of candidates.

**B. Define the Design Objective**

- Measurable objective 1: return exactly the same optimum value as the exhaustive search on every test instance (correctness is not negotiable).
- Measurable objective 2: examine at most 1% of the 4,194,304 candidates at n = 22 (<= 41,943) by never examining prefixes that are already overweight and by cutting subtrees that cannot beat the best value found so far (reduce the number of candidate solutions / repeated processing).
- Measurable objective 3: finish n = 22 in well under 1 s (original 1.6 s) and extend the deployable size beyond the n = 22 limit of Q6 (response limit 5 s).

**C. Design the Revised Computational Strategy**

- **What has changed:** the search is still a depth-first include/exclude enumeration, but (1) it tests feasibility at EVERY node, not only at the leaves - an overweight prefix is never extended; (2) it computes an optimistic upper bound for what a prefix can still achieve and abandons the prefix if the bound cannot beat the best value found so far; (3) items are processed in descending value/weight order so that good solutions are found early and the bound is tight.
- **What information or property is exploited:** (a) monotonic weight - adding items never reduces weight, so overweight prefixes are dead ends; (b) value density ordering - if the remaining capacity could even be filled perfectly with fractional items of the best remaining density, the value is an upper bound for any completion (a relaxation that can only overestimate); (c) an incumbent (best known) value that rises early in a greedy-first dive.
- **Operations avoided or reduced:** generation of the 2^(n-i) completions of overweight prefixes; full evaluation of prefixes whose bound <= incumbent. Candidate leaves examined fall from 2^n to a few.

- **Preprocessing introduced:** yes - one sort of the packages by value/weight, O(n log n), negligible versus the search.
- **Additional memory:** O(n) (recursion stack and order array) - no table is stored.
- **New trade-offs:** each node now costs O(n) for the bound instead of O(1); the improvement depends on the data (if all items have the same density the bound cannot prune - tested in part E) and the worst case remains exponential.

```
ALGORITHM PrunedAirlift(w, v, C)
sort packages by v[i]/w[i] descending                    // preprocessing, O(n log n)
best <- 0
procedure Search(i, weight, value)                       // node = a partial selection of packages 1..i-1
   nodes <- nodes + 1
   if value > best then best <- value ; record selection
   if i > n then return
   if UpperBound(i, weight, value) <= best then return   // cannot beat incumbent: prune subtree
   if weight + w[i] <= C then Search(i+1, weight + w[i], value + v[i])   // include only if still feasible
   Search(i+1, weight, value)                                  // exclude

function UpperBound(i, weight, value)                    // optimistic value of any completion
   room <- C - weight ; b <- value
   for k <- i to n do
      if w[k] <= room then room <- room - w[k] ; b <- b + v[k]
      else b <- b + v[k] * room / w[k] ; break           // take a fraction of the next package
   return b
```

### D. Predict Before Implementing

- Prediction: the redesigned search returns the same optimal value as the original on every instance (it only discards candidates that are infeasible or cannot beat the best).
- Performance measure expected to improve: the number of candidate solutions / search nodes examined - from 2^n to orders of magnitude fewer on typical random instances, with the node count growing far more slowly than 2^n; execution time should fall in step for larger n, although at very small n (<= ~12) the per-node bound overhead may make it no faster.
- Expected limit: on instances where the bound is uninformative (all items with identical value-to-weight ratio and an unattainable exact fill) pruning by bound should fail and the improvement should largely disappear.

### E. Implement and Validate the Redesign

```
def bnb_knapsack(W, V, cap):
   n = len(W)
   order = sorted(range(n), key=lambda i: V[i]/W[i], reverse=True)       # preprocessing
   w = [W[i] for i in order]; v = [V[i] for i in order]
   stats = {"nodes": 0, "pruned_weight": 0, "pruned_bound": 0, "leaves": 0}
   best = [0, None]; chosen = [0]*n
   def bound(i, wt, val):
      room = cap - wt; b = val
      for k in range(i, n):
         if w[k] <= room: room -= w[k]; b += v[k]
         else: b += v[k]*room/w[k]; break
      return b
   def rec(i, wt, val):
      stats["nodes"] += 1
      if val > best[0]: best[0] = val; best[1] = chosen[:]
      if i == n: stats["leaves"] += 1; return
      if bound(i, wt, val) <= best[0]: stats["pruned_bound"] += 1; return
```

```
    if wt + w[i] <= cap: chosen[i] = 1; rec(i+1, wt + w[i], val + v[i]); chosen[i] = 0
    else: stats["pruned_weight"] += 1
    rec(i+1, wt, val)
rec(0, 0, 0) ...
```

Same instances (same weights, values, payload limit) as in Q6; the optimum of the redesign was asserted equal to the exhaustive optimum on every instance, and its selection was checked for feasibility and value.

| n | Original: candidates (2^n) | Redesign: nodes | Reduction | Same optimum | Original time (s) | Redesign time (s) |
|---|---|---|---|---|---|---|
| 8 | 256 | 9 | 28x | 239 | 0.0001 | 0.00005 |
| 10 | 1,024 | 16 | 64x | 269 | 0.0003 | 0.00003 |
| 12 | 4,096 | 92 | 45x | 442 | 0.0014 | 0.00007 |
| 14 | 16,384 | 22 | 745x | 514 | 0.0058 | 0.00003 |
| 16 | 65,536 | 44 | 1,489x | 570 | 0.0222 | 0.00004 |
| 18 | 262,144 | 123 | 2,131x | 566 | 0.0901 | 0.00012 |
| 20 | 1,048,576 | 107 | 9,800x | 654 | 0.5323 | 0.00008 |
| 22 | 4,194,304 | 102 | 41,121x | 917 | 1.9885 | 0.00008 |

| n (beyond Q6 limit) | Redesign nodes | Redesign time (s) | Exhaustive would need (predicted) |
|---|---|---|---|
| 26 | 140 | 0.0001 | 31.8 s |
| 30 | 51 | 0.0001 | 509 s |
| 40 | 244 | 0.0002 | 5.21e+05 s |
| 60 | 137 | 0.0003 | 5.47e+11 s |
| 100 | 213 | 0.0004 | 6.01e+23 s |

**Stress test where pruning should fail (equal value density: value = weight, even weights, odd payload limit so no exact fill exists - the bound can never rule a prefix out):**

| n | Redesign nodes | Full tree (2^(n+1)-1) | % of full tree | Bound prunes | Exhaustive time (s) | Redesign time (s) |
|---|---|---|---|---|---|---|
| 14 | 9,379 | 32,767 | 28.6% | 1,679 | 0.0058 | 0.0056 |
| 16 | 38,007 | 131,071 | 29.0% | 2,595 | 0.0250 | 0.0222 |
| 18 | 107,126 | 524,287 | 20.4% | 14,276 | 0.1126 | 0.0901 |
| 20 | 587,629 | 2,097,151 | 28.0% | 76,270 | 0.3995 | 0.3585 |
| 22 | 2,159,087 | 8,388,607 | 25.7% | 317,912 | 1.8959 | 2.1095 |

### F. Defend the Final Design

- **Was the prediction supported?** Yes, in three parts. (1) Same optimum on all instances (assertions). (2) Candidates/nodes fell by orders of magnitude: n = 22 needs 102 nodes instead of 4,194,304 candidates (41,121x fewer), and the node count stays roughly flat (9-123 nodes for n = 8-22; it varies between instances because each size uses a different random instance). (3) The predicted limit appeared: on the equal-density stress instances the redesign still examined 25.7% of the full search tree at n = 22 (2,159,087 nodes) and took 2.11 s against 1.90 s for the original - no improvement, and slightly slower because of the bound computation.

- **Magnitude of improvement:** n = 22: 1988 ms -> 0.08 ms (~24,121x); n = 100 is solved in 0.42 ms where the exhaustive search would need ~10^16 years; the deployable size rises from 22 to at least 100 packages on typical (random) data.
- **New costs / trade-offs:** O(n) bound computation per node, an O(n log n) sort (the small-n overhead I predicted in part D was NOT observed: even at n = 8 the redesign took 0.05 ms against 0.09 ms - a prediction that was not supported, because the pruned tree is so small that the bound cost is negligible), more complex code to verify, and performance that depends on the data rather than only on n.
- **When the original may still be preferable:** tiny instances (n <= ~12, microseconds either way) where simplicity and ease of verification matter; as an independent correctness oracle for testing the redesign (as used here); and for adversarial or equal-density data where pruning gives no benefit and the redesign is no faster.
- **Does it sufficiently address the identified limitation?** Partly, and honestly so: it removes the diagnosed wasted work (overweight prefixes and dominated subtrees) and makes typical airlift instances (random weights/values) solvable far beyond n = 22, but the worst case is still exponential (equal-density data: 25.7% of 2^(n+1) nodes at n = 22). For deployment I would therefore keep a time limit with the best-found loading as a fallback, and keep the exhaustive search for n <= 12 and for verification.
- **Conclusion:** the redesign met objectives 1-3 on typical data (same optimum; 0.002% of the candidates at n = 22; ~17,000x faster), and its failure on the engineered equal-density case is explained by the diagnosis: the bound exploits differences in value density, and when none exist there is nothing to prune on.